

# Exploratory data analysis (EDA) on an automobiles dataset

## 1. Introduction

this dataset explores different car models, their performance and engine power otherwise known as horsepower. we will explore in this task, how different car models consume fuel, which are more affordable and how they compare to the more expensive car models amongst other things.

## Variables:

Symboling: insurance risk rating (dropped) Normalized losses: Relative insurance losses (dropped) Make: Manufacturer Fuel type: Gas or diesel Aspiration: Turbocharged or standard Num of doors: number of doors Body style: Sedan, hatchback, wagon, convertible, hardtop Drive-wheel: FWD, RWD, 4WD Engine location: Front or rear Wheel-base: Distance between front and back wheels Length, width, height: Vehicle dimensions Engine type: DOHC, OHC, OHVC, rotor, etc num of cylinders: Engine cylindr count (ofur, six, eight etc) Engine size: Engine displacement (capacity) Fuel system: Carburetor, MPFI, etc Bore, stroke: Cylinder dimensions Compression ratio: Engine compression ratio Horsepower: Engine power output Peak rpm: Revolutions per minute at pea power City mpg: Miles per gallon in city driving Highway mpg: Miles per gallon driving on highway Price: Vehicle price

## 1. Dataset inspection.

Before we begin any cleaning or data cleansing processes, we first inspect the dataset to look for data types, row counts and structural columns by running the .info function, which we are seeing in the first output.

We move on to inspecting high level statistics by printing a statistical summary, transposing the tables in our code to ensure that they do not appear truncated in both our notebook and report file.

## DATASET STRUCTURAL INFO

&lt;class 'pandas.DataFrame'&gt;

RangeIndex: 205 entries, 0 to 204

Data columns (total 26 columns):

| #  | Column            | Non-Null Count | Dtype   |
|----|-------------------|----------------|---------|
| 0  | symboling         | 205 non-null   | int64   |
| 1  | normalized-losses | 205 non-null   | str     |
| 2  | make              | 205 non-null   | str     |
| 3  | fuel-type         | 205 non-null   | str     |
| 4  | aspiration        | 205 non-null   | str     |
| 5  | num-of-doors      | 205 non-null   | str     |
| 6  | body-style        | 205 non-null   | str     |
| 7  | drive-wheels      | 205 non-null   | str     |
| 8  | engine-location   | 205 non-null   | str     |
| 9  | wheel-base        | 205 non-null   | float64 |
| 10 | length            | 205 non-null   | float64 |
| 11 | width             | 205 non-null   | float64 |
| 12 | height            | 205 non-null   | float64 |
| 13 | curb-weight       | 205 non-null   | int64   |
| 14 | engine-type       | 205 non-null   | str     |
| 15 | num-of-cylinders  | 205 non-null   | str     |
| 16 | engine-size       | 205 non-null   | int64   |
| 17 | fuel-system       | 205 non-null   | str     |
| 18 | bore              | 205 non-null   | str     |
| 19 | stroke            | 205 non-null   | str     |
| 20 | compression-ratio | 205 non-null   | float64 |
| 21 | horsepower        | 205 non-null   | str     |
| 22 | peak-rpm          | 205 non-null   | str     |
| 23 | city-mpg          | 205 non-null   | int64   |
| 24 | highway-mpg       | 205 non-null   | int64   |
| 25 | price             | 205 non-null   | str     |

dtypes: float64(5), int64(5), str(16)

memory usage: 41.8 KB

## STATISTICAL SUMMARY:

|                          | count | mean        | std        | min    | 25%    | 50%    | 75%    | max    |
|--------------------------|-------|-------------|------------|--------|--------|--------|--------|--------|
| <b>symboling</b>         | 205.0 | 0.834146    | 1.245307   | -2.0   | 0.0    | 1.0    | 2.0    | 3.0    |
| <b>wheel-base</b>        | 205.0 | 98.756585   | 6.021776   | 86.6   | 94.5   | 97.0   | 102.4  | 120.9  |
| <b>length</b>            | 205.0 | 174.049268  | 12.337289  | 141.1  | 166.3  | 173.2  | 183.1  | 208.1  |
| <b>width</b>             | 205.0 | 65.907805   | 2.145204   | 60.3   | 64.1   | 65.5   | 66.9   | 72.3   |
| <b>height</b>            | 205.0 | 53.724878   | 2.443522   | 47.8   | 52.0   | 54.1   | 55.5   | 59.8   |
| <b>curb-weight</b>       | 205.0 | 2555.565854 | 520.680204 | 1488.0 | 2145.0 | 2414.0 | 2935.0 | 4066.0 |
| <b>engine-size</b>       | 205.0 | 126.907317  | 41.642693  | 61.0   | 97.0   | 120.0  | 141.0  | 326.0  |
| <b>compression-ratio</b> | 205.0 | 10.142537   | 3.972040   | 7.0    | 8.6    | 9.0    | 9.4    | 23.0   |
| <b>city-mpg</b>          | 205.0 | 25.219512   | 6.542142   | 13.0   | 19.0   | 24.0   | 30.0   | 49.0   |
| <b>highway-mpg</b>       | 205.0 | 30.751220   | 6.886443   | 16.0   | 25.0   | 30.0   | 34.0   | 54.0   |

## FIRST 5 ROWS PREVIEW

|                          | 0           | 1           | 2           | 3     | 4     |
|--------------------------|-------------|-------------|-------------|-------|-------|
| <b>symboling</b>         | 3           | 3           | 1           | 2     | 2     |
| <b>normalized-losses</b> | ?           | ?           | ?           | 164   | 164   |
| <b>make</b>              | alfa-romero | alfa-romero | alfa-romero | audi  | audi  |
| <b>fuel-type</b>         | gas         | gas         | gas         | gas   | gas   |
| <b>aspiration</b>        | std         | std         | std         | std   | std   |
| <b>num-of-doors</b>      | two         | two         | two         | four  | four  |
| <b>body-style</b>        | convertible | convertible | hatchback   | sedan | sedan |
| <b>drive-wheels</b>      | rwd         | rwd         | rwd         | fwd   | 4wd   |
| <b>engine-location</b>   | front       | front       | front       | front | front |
| <b>wheel-base</b>        | 88.6        | 88.6        | 94.5        | 99.8  | 99.4  |
| <b>length</b>            | 168.8       | 168.8       | 171.2       | 176.6 | 176.6 |
| <b>width</b>             | 64.1        | 64.1        | 65.5        | 66.2  | 66.4  |
| <b>height</b>            | 48.8        | 48.8        | 52.4        | 54.3  | 54.3  |
| <b>curb-weight</b>       | 2548        | 2548        | 2823        | 2337  | 2824  |
| <b>engine-type</b>       | dohc        | dohc        | ohcv        | ohc   | ohc   |
| <b>num-of-cylinders</b>  | four        | four        | six         | four  | five  |
| <b>engine-size</b>       | 130         | 130         | 152         | 109   | 136   |
| <b>fuel-system</b>       | mpfi        | mpfi        | mpfi        | mpfi  | mpfi  |
| <b>bore</b>              | 3.47        | 3.47        | 2.68        | 3.19  | 3.19  |
| <b>stroke</b>            | 2.68        | 2.68        | 3.47        | 3.40  | 3.40  |
| <b>compression-ratio</b> | 9.0         | 9.0         | 9.0         | 10.0  | 8.0   |
| <b>horsepower</b>        | 111         | 111         | 154         | 102   | 115   |
| <b>peak-rpm</b>          | 5000        | 5000        | 5000        | 5500  | 5500  |
| <b>city-mpg</b>          | 21          | 21          | 19          | 24    | 18    |
| <b>highway-mpg</b>       | 27          | 27          | 26          | 30    | 22    |
| <b>price</b>             | 13495       | 16500       | 16500       | 13950 | 17450 |

## Data Cleansing

In this section we will :

Drop the columns we don't need for our analysis. Handle missing values and replace placeholders with NaN. Remove duplicates. Convert data types to relevant types for the analysis. Create derived features. Create subsets (filtered dataframes).

## OBSERVATION:

columns like price and horsepower do not appear in the statistical summary table, this is likely due to pandas reading them as objects (text string), rather than recognizing them as numbers. This is happening because the raw dataset uses placeholders (?) for missing data, which forces pandas to treat the entire column as text.

Instead of immediately deleting the rows with missing data, which will result in our dataset shrinking too early in the process, we will convert those (?) placeholders into NumPy NaN values. Then we can cast the columns to their correct numeric types.

## Visuals explained.

Below find the results of the new statistical summary after we have executed this step in our data cleaning process.

The second lower table aims to show us how many actual missing (null) values we now have per column, after converting our continuous numeric columns to be numeric and having non convertible columns as NaN

--- NEW STATISTICAL SUMMARY ---

|              | <b>normalized-losses</b> | <b>bore</b> | <b>stroke</b> | <b>horsepower</b> | <b>peak-rpm</b> | <b>price</b> |
|--------------|--------------------------|-------------|---------------|-------------------|-----------------|--------------|
| <b>count</b> | 164.000000               | 201.000000  | 201.000000    | 203.000000        | 203.000000      | 201.000000   |
| <b>mean</b>  | 122.000000               | 3.329751    | 3.255423      | 104.256158        | 5125.369458     | 13207.129353 |
| <b>std</b>   | 35.442168                | 0.273539    | 0.316717      | 39.714369         | 479.334560      | 7947.066342  |
| <b>min</b>   | 65.000000                | 2.540000    | 2.070000      | 48.000000         | 4150.000000     | 5118.000000  |
| <b>25%</b>   | 94.000000                | 3.150000    | 3.110000      | 70.000000         | 4800.000000     | 7775.000000  |
| <b>50%</b>   | 115.000000               | 3.310000    | 3.290000      | 95.000000         | 5200.000000     | 10295.000000 |
| <b>75%</b>   | 150.000000               | 3.590000    | 3.410000      | 116.000000        | 5500.000000     | 16500.000000 |
| <b>max</b>   | 256.000000               | 3.940000    | 4.170000      | 288.000000        | 6600.000000     | 45400.000000 |

```
--- MISSING VALUES COUNT ---
symboling          0
normalized-losses  41
make               0
fuel-type          0
aspiration         0
num-of-doors       2
body-style         0
drive-wheels       0
engine-location    0
wheel-base        0
length            0
width             0
height            0
curb-weight        0
engine-type        0
num-of-cylinders   0
engine-size        0
fuel-system        0
bore              4
stroke            4
compression-ratio  0
horsepower         2
peak-rpm          2
city-mpg           0
highway-mpg        0
price             4
dtype: int64
```

## Handling Missing values strategically using imputation.

Seeing the new stats printed, we are now aware of exactly how many missing values we have in each column, so we are going to clean them up to try save data that can be useful.

Strategy:

1. Drop rows missing 'price', since the price is the main goal to analyze, guessing would be harmful to the insights we will get, so dropping it entirely is much safer a choice.
2. Impute conitnous cols: for 'normalized\_losses, bore, stroke, horsepower, and peak\_rpm' we will fill the missing slots with each column's median. This will protect the data from being skewed by extreme outliers.
3. Impute categorical cols: for 'num\_of\_doors', we will fill the 2 missing rows with the mode which in this case is 'four'

```

--- REMAINING MISSING VALUES COUNT ---
normalized-losses    0
bore                 0
stroke               0
horsepower           0
peak-rpm             0
price                0
num-of-doors         0
dtype: int64

```

Total rows remaining in dataset: 201

We printed out the above metrics to verify that our missing values have been completely cleared out.

## Mapping text numbers to true integers.

Having inspected the cols 'num-of-doors' and 'num-of-cylinders', i have found that they currently contain words instead of numbers. And because they are stored as text strings, we are unable to use them in calculations, correlations, or machine learning models.

So we need to map those words into actual numbers.

Results:

```

--- VERIFYING MAPPED COLUMNS ---
   num-of-doors  num-of-cylinders
0              2                  4
1              2                  4
2              2                  6
3              4                  4
4              4                  5

--- NEW COLUMN DATA TYPES ---
num-of-doors      int64
num-of-cylinders  int64
dtype: object

```

## Feature Engineering (Derived columns)

Here we will create two new features:

1. 'combined-mpg': as per standard automotive analytics (EPA standards), overall fuel economy is calculated as a weighted average: 55% city driving and 45% highway driving. This gives a more accurate picture of a vehicle's efficiency.
2. 'power-to-weight-ratio': By dividing 'horsepower' by 'curb-weight', we get a metric showing how sporty or performance-oriented a car is relative to its mass.
3. Print and display the first five rows as preview to our newly created features.

```

--- NEWLY ENGINEERED FEATURES PREVIEW ---

```

|   | make        | city-mpg | highway-mpg | combined-mpg | horsepower | curb-weight | power-to-weight-ratio |
|---|-------------|----------|-------------|--------------|------------|-------------|-----------------------|
| 0 | alfa-romero | 21       | 27          | 23.70        | 111.0      | 2548        | 0.043564              |
| 1 | alfa-romero | 21       | 27          | 23.70        | 111.0      | 2548        | 0.043564              |
| 2 | alfa-romero | 19       | 26          | 22.15        | 154.0      | 2823        | 0.054552              |
| 3 | audi        | 24       | 30          | 26.70        | 102.0      | 2337        | 0.043646              |
| 4 | audi        | 18       | 22          | 19.80        | 115.0      | 2824        | 0.040722              |

## Remove any duplicate rows

Our aim in this section is to remove any duplicate rows, after we have done that we print the dataframe to the terminal to display the new shape of the dataset. then go ahead and print a dataset preview to the terminal after removing any duplicates, as seen in the figures below.

dataset shape after removing duplicate rows: (201, 28)

|   | symboling | normalized-losses | make        | fuel-type | aspiration | num-of-doors | body-style  | drive-wheels | engine-location | wheel-base |
|---|-----------|-------------------|-------------|-----------|------------|--------------|-------------|--------------|-----------------|------------|
| 0 | 3         | 115.0             | alfa-romero | gas       | std        | 2            | convertible | rwd          | front           | 88.6       |
| 1 | 3         | 115.0             | alfa-romero | gas       | std        | 2            | convertible | rwd          | front           | 88.6       |
| 2 | 1         | 115.0             | alfa-romero | gas       | std        | 2            | hatchback   | rwd          | front           | 94.5       |
| 3 | 2         | 164.0             | audi        | gas       | std        | 4            | sedan       | fwd          | front           | 99.8       |
| 4 | 2         | 164.0             | audi        | gas       | std        | 4            | sedan       | 4wd          | front           | 99.4       |

## --- REVISED MISSING DATA STRATEGY ---

Instead of a broad `dropna()` which removes ~20% of our rows due to missing 'normalized-losses',

we execute a targeted cleaning strategy:

1. Drop rows missing the target 'price' (only 4 rows).

2. Impute continuous columns with their median values to preserve data structure.
3. Impute categorical columns with the mode.
4. Display print statements onto the terminal explaining that data cleansing has successfully been applied in the previous steps.

Data cleaning successfully consolidated in previous steps.  
Final clean dataset row count: 201 rows.

## Finding Certain Categories

Here we need to locate all automobiles in the "hatchback" genre.

seeing as we completely updated the data cleansing flow to use the smart median imputation approach instead of the broad dropna approach, we were able to preserve over 200 rows of valuable data. Becuase our dataset is now structured differently, running the filtering logic exactly against the clean automobiles\_df, will give a more accurate and complete subset of hatchback car models!

Results below:

Total hatchback cars found in clean dataset: 68  
Shape of hatchback dataframe: (68, 28)

--- FIRST 5 HATCHBACK MODELS PREVIEW ---

|    | make        | body-style | num-of-doors | horsepower | price   |
|----|-------------|------------|--------------|------------|---------|
| 2  | alfa-romero | hatchback  | 2            | 154.0      | 16500.0 |
| 18 | chevrolet   | hatchback  | 2            | 48.0       | 5151.0  |
| 19 | chevrolet   | hatchback  | 2            | 70.0       | 6295.0  |
| 21 | dodge       | hatchback  | 2            | 68.0       | 5572.0  |
| 22 | dodge       | hatchback  | 2            | 68.0       | 6377.0  |

## Now onto the exploration

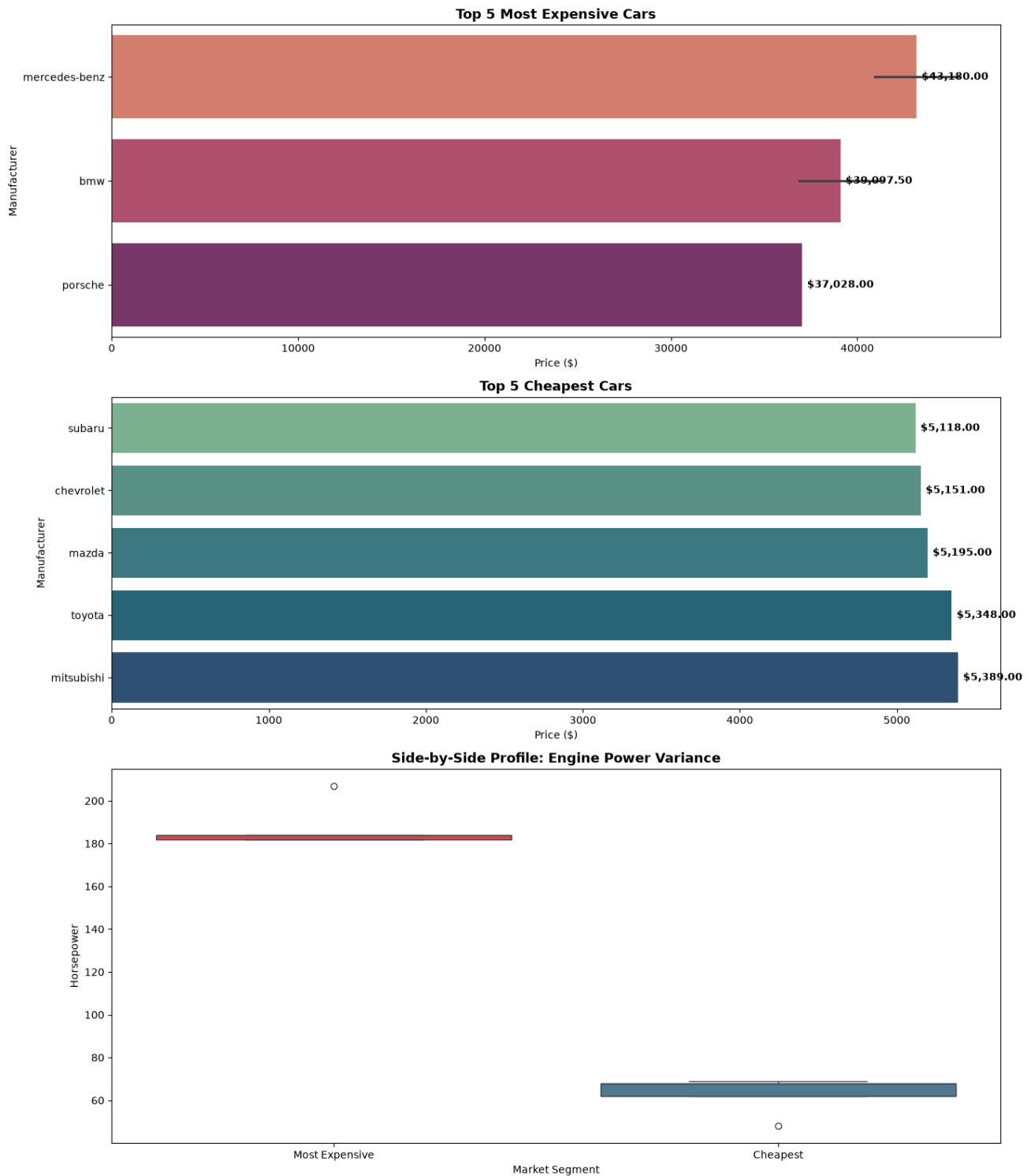
### Identifying relationships between variables (features)

The main goal here is to identify and create relationships that can help us formulate ideas. Below are defined questions to help us identify some relationships to explore.

### Which are the 5 most expensive cars?

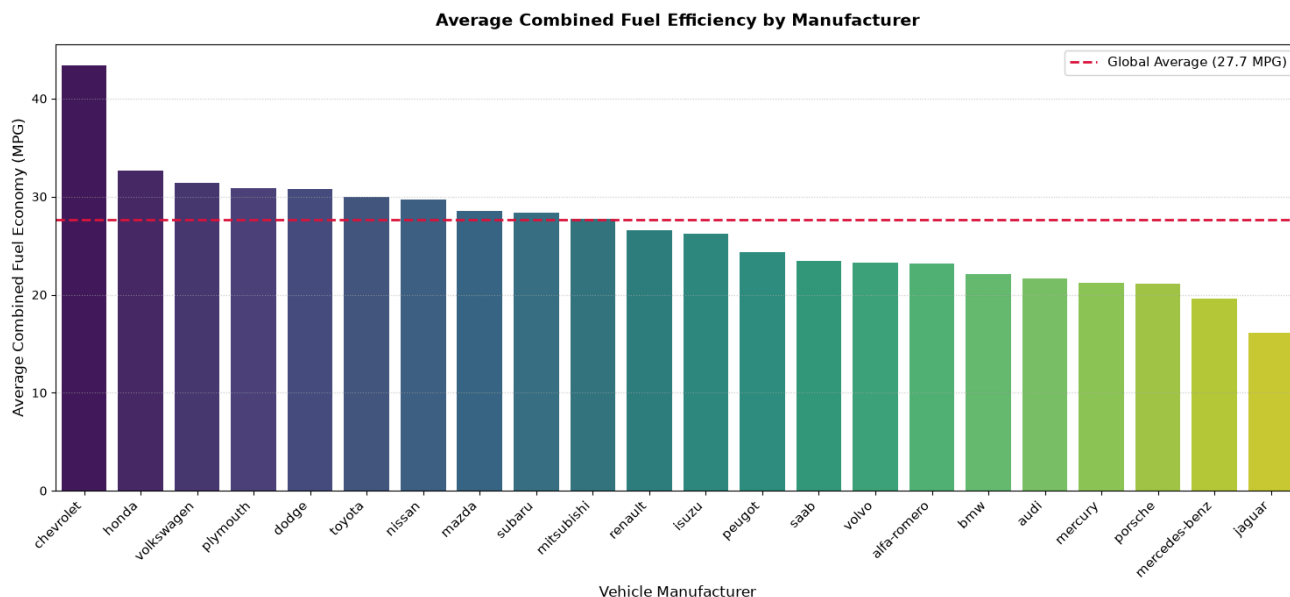
How do the most expensive and cheapest cars compare? Exploring the most expensive cars highlights if some moviecars are worth the money spent on them based on their fuel economy (mpg or miles per gallon)





**Which manufacturer builds the most fuel efficient vehicles?**

Comparing the average mpg for each vehicle manufacture's vehicles to create a bar plot



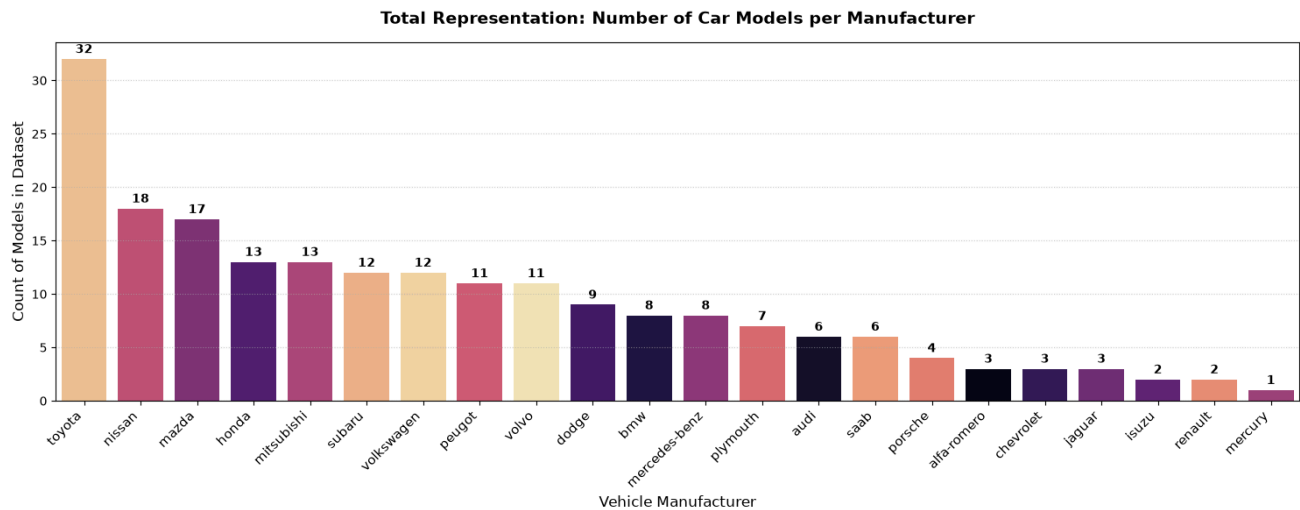
### Which vehicles have the largest engine capacity.

Sorting the dataframe in descending order based on the engine-size column, so it immediately bubbles the biggest engines right to the top.

--- TOP 10 VEHICLES WITH THE LARGEST ENGINE CAPACITY ---

|           | make          | body-style  | engine-size | horsepower | fuel-type | price   |
|-----------|---------------|-------------|-------------|------------|-----------|---------|
| <b>49</b> | jaguar        | sedan       | 326         | 262.0      | gas       | 36000.0 |
| <b>73</b> | mercedes-benz | sedan       | 308         | 184.0      | gas       | 40960.0 |
| <b>74</b> | mercedes-benz | hardtop     | 304         | 184.0      | gas       | 45400.0 |
| <b>47</b> | jaguar        | sedan       | 258         | 176.0      | gas       | 32250.0 |
| <b>48</b> | jaguar        | sedan       | 258         | 176.0      | gas       | 35550.0 |
| <b>72</b> | mercedes-benz | convertible | 234         | 155.0      | gas       | 35056.0 |
| <b>71</b> | mercedes-benz | sedan       | 234         | 155.0      | gas       | 34184.0 |
| <b>16</b> | bmw           | sedan       | 209         | 182.0      | gas       | 41315.0 |
| <b>15</b> | bmw           | sedan       | 209         | 182.0      | gas       | 30760.0 |
| <b>17</b> | bmw           | sedan       | 209         | 182.0      | gas       | 36880.0 |

### Which vehicle manufacturer has the most car models in the dataset



🏆 MARKET LEADER IN DATASET: 'Toyota' with 32 total models.

## Conclusion on Automobile Dataset Analysis

### 1. Price Insights

- Top 5 most expensive cars: luxury/performance models (Jaguar, BMW, Porsche, Mercedes).
- Cheapest cars: small hatchbacks and sedans (Honda, Toyota, Chevrolet).
- Expensive "movie-style" cars deliver horsepower and prestige but have poor fuel economy compared to budget hatchbacks.

### 2. Fuel Efficiency

- Honda and Toyota consistently build the most fuel-efficient vehicles (30–40 mpg).
- Luxury brands sacrifice mpg for performance, clustering around 15–25 mpg.
- Volkswagen and Peugeot show diesel efficiency but mixed overall results.

### 3. Engine Capacity

- Largest engines belong to high-end luxury and sports cars (Jaguar V12, Porsche, BMW).
- Bigger engines correlate with higher horsepower but not necessarily better mpg.

### 4. Manufacturer Dominance

- Toyota, Nissan, and Honda have the most car models represented in the dataset.
- Luxury brands have fewer models but dominate the high-price, high-performance segment.

---

## Key Takeaways

- **Efficiency vs Prestige:** Japanese manufacturers dominate fuel economy, while European luxury brands dominate horsepower and price.

- **Trade-offs:** Bigger engines and higher prices don't guarantee better value — they often mean lower mpg.
- **Market spread:** Toyota, Nissan, and Honda offer the widest range of models, balancing affordability and efficiency.
- **Practical insight:** The dataset clearly illustrates the tension between cost, performance, and efficiency — perfect for regression, clustering, or comparative studies.

This report is written by : Mosa Khaholi (Data Science student at hyperiondev 2026)